

Machine Learning for Microbial Metagenomics

A guided lab using mikropml, caret, and intentional error diagnostics

NBIB25004U ML Practical

2026-05-07

What You Will Build

By the end of this practical, you will have run a complete classification workflow:

1. Prepare microbiome feature data without leaking information.
2. Train logistic regression and random forest classifiers.
3. Evaluate models using AUC, precision-recall, and accuracy.
4. Tune hyperparameters and compare default versus tuned models.
5. Interpret models with permutation importance and coefficients.
6. Diagnose common incorrect data preparation signatures.

Dataset snapshot:

Item	Value
Samples	200
OTU features	10
Outcome	dx
Cancer	98
Normal	102

Repo Layout

```
1 Week_21
2 |— config.R                # seed, built-in data, output paths
3 |— 01_data_preparation.R   # correct structural cleaning
4 |— 01b_data_preparation_witherrors.R # intentional diagnostic errors
5 |— 02_model_training.R     # train glmnet and random forest
6 |— 03_model_evaluation.R   # metrics, ROC, PRC, comparison plots
7 |— 04_hyperparameter_tuning.R # tune lambda and mtry
8 |— 05_feature_importance.R # permutation importance and coefficients
9 |— results/               # .rds and .csv outputs
10 |— figures/              # generated figures
11 |— logs/                 # script logs
```

Run scripts from [Week_21](#). Each numbered script reads the previous stage's output.

Why Machine Learning for Metagenomics?

Metagenomic datasets often have:

- many microbial features relative to samples
- correlated taxa or OTUs
- noisy measurements and biological heterogeneity
- clinically meaningful labels such as disease status, treatment response, or environment

Machine learning helps ask:

- Can microbial profiles predict a phenotype?
- Which taxa contribute most to prediction?
- Does a model generalize to samples it did not see during training?

Prediction is useful only if the evaluation is honest. Most of this lab is about avoiding over-optimistic results.

Why `mikropml`?

`mikropml` is a microbiome-focused interface around common R machine learning tools, especially `caret`.

- `caret` provides model training, cross-validation, preprocessing, and prediction APIs.
- `mikropml` provides defaults and helper functions that are convenient for microbial classification.
- The package lets beginners run rigorous workflows while still learning concepts that transfer to direct `caret` workflows later.

```
1 # config.R lines 9-11
2 library(mikropml)
3 data("otu_mini_bin")
4 raw_data <- otu_mini_bin
```

Local package version used for this deck: `mikropml` 1.6.2.

Run Order and What to Inspect

```
1 Rscript 01_data_preparation.R
2 Rscript 02_model_training.R
3 Rscript 03_model_evaluation.R
4 Rscript 04_hyperparameter_tuning.R
5 Rscript 05_feature_importance.R
6
7 # Optional diagnostic lab; do not feed its outputs into scripts 02-05.
8 Rscript 01b_data_preparation_witherrors.R
```

Stage	Key output	Inspect
01	results/prepared_data.rds	dimensions, labels, feature types
02	results/model_results.rds	train/test AUC
03	performance_comparison.csv , ROC/PRC plots	model behavior
04	tuning_comparison_summary.csv	default vs tuned
05	feature_importance.rds , plots	biological interpretation
01b	witherrors_* outputs	failure signatures

Error Types This Workflow Controls

Error type	Characteristic signature	Correct workflow response
Full-data preprocessing leakage	scaling/filtering used test rows	learned preprocessing happens in <code>caret::train()</code>
Outcome leakage	feature nearly perfectly predicts <code>dx</code>	scan suspicious features; remove label-derived columns
Bad outcome conversion	<code>dx</code> becomes <code>NA</code> or numeric	convert only feature columns
Broken sample-label alignment	signal collapses or labels mismatch rows	keep rows and labels together
Overfitting	high CV AUC but poor test AUC	hold-out test set and cross-validation

```
1 # 02_model_training.R lines 42-45
2 model_preprocess <- c("nzv", "corr", "center", "scale")
```

Script 01: Correct Data Preparation

Goal: perform only structural data hygiene before model training.

```
1 # 01_data_preparation.R lines 33-47
2 cleaned_data <- raw_data %>%
3   clean_names()
4
5 if (!"dx" %in% names(cleaned_data)) {
6   stop("Error: expected outcome column `dx` was not found.")
7 }
8
9 cleaned_data <- cleaned_data %>%
10   mutate(across(-dx, as.numeric))
```

What this does:

- makes column names R-safe with `janitor::clean_names()`
- confirms the outcome column exists
- converts OTU columns to numeric
- does not scale, filter, or learn from distributions yet

Script 01: What Happens to the Data?

Check	Expected result
Rows	200
Columns	11
Features	10 OTUs
Outcome levels	<code>cancer, normal</code>
Cancer samples	98
Normal samples	102

Inspect after running:

File	What it should show
<code>logs/01_data_preparation.log</code>	sample and class counts
<code>results/prepared_data.rds</code>	structurally clean data

Red flags:

- `dx` missing or renamed
- features stored as character
- full-dataset scaling before split

```
1 # 01_data_preparation.R lines 49-62
2 # Learned preprocessing is deferred to model training.
3 output_file <- file.path(results_dir, "prepared_data.rds")
4 saveRDS(cleaned_data, file = output_file)
```

Mathematically, no feature means, standard deviations, correlations, or variance filters are estimated in 01.

Script 02: Training Design

`run_ml()` handles train/test partitioning and cross-validation.

```
1 # 02_model_training.R lines 35-45
2 dat <- readRDS(file.path(results_dir, "prepared_data.rds"))
3
4 model_preprocess <- c("nzv", "corr", "center", "scale")
```

```
1 # 02_model_training.R lines 56-63
2 results_glmnet <- run_ml(
3   dataset = dat,
4   method = "glmnet",
5   outcome_colname = "dx",
6   seed = seed,
7   preProcess = model_preprocess,
8   cv_times = 2
9 )
```

`preProcess` is passed into `caret::train()`, so scaling and filtering are estimated inside training/resampling.

Script 02: Two Model Families

Model	In this lab	Intuition
<code>glmnet</code>	regularized logistic regression	baseline linear classifier
<code>rf</code>	random forest	nonlinear ensemble of decision trees

```
1 # 02_model_training.R lines 69-75
2 results_rf <- run_ml(
3   dataset = dat,
4   method = "rf",
5   outcome_colname = "dx",
6   seed = seed,
7   preProcess = model_preprocess,
8   cv_times = 2
9 )
```

Qualitatively:

- logistic regression fits weighted feature effects
- random forest averages many decision trees
- cross-validation estimates how well settings work on unseen folds

Script 02: Training Outputs

Current default model results:

Method	CV AUC	Test AUC	prAUC	Accuracy
glmnet	0.625	0.634	0.625	0.513
rf	0.658	0.696	0.642	0.667

```
1 # 02_model_training.R lines 80-83
2 results_glmnet$performance$cv_metric_AUC
3 results_glmnet$performance$AUC
4 results_rf$performance$cv_metric_AUC
5 results_rf$performance$AUC
```

If CV AUC is excellent but test AUC is poor, suspect overfitting or leakage.

Script 03: Evaluation Metrics

```
1 # 03_model_evaluation.R lines 44-56
2 perf_glmnet <- fix_perf_types(model_list$glmnet$performance)
3 perf_rf <- fix_perf_types(model_list$rf$performance)
4 combined_perf <- bind_rows(perf_glmnet, perf_rf)
5 write.csv(combined_perf, "results/performance_comparison.csv")
```

Metric	Meaning
AUC	ranking ability across thresholds
prAUC	precision-recall behavior, useful when classes are imbalanced
Accuracy	fraction of samples classified correctly
Sensitivity	true positive rate
Specificity	true negative rate
F1	balance of precision and recall

ROC and AUC: Why They Matter

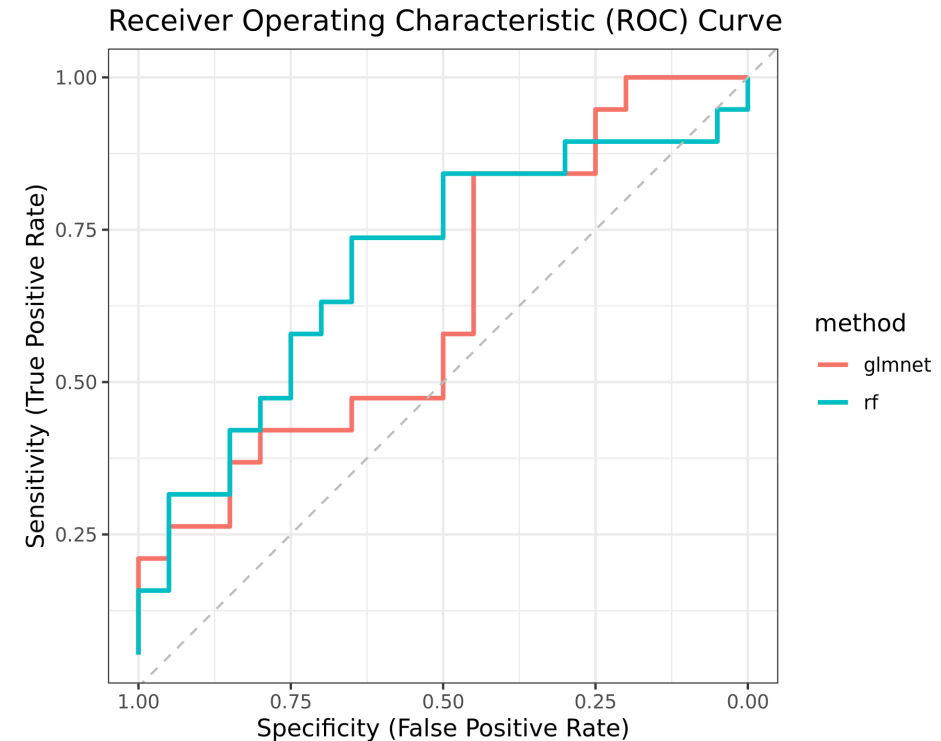
```
1 # 03_model_evaluation.R lines 80-103
2 calc_model_sensspec(result$trained_model, result$test_data, "dx")
3 ggplot(aes(x = specificity, y = sensitivity, color = method)) +
4   geom_line(linewidth = 1) +
5   scale_x_reverse()
```

ROC curve:

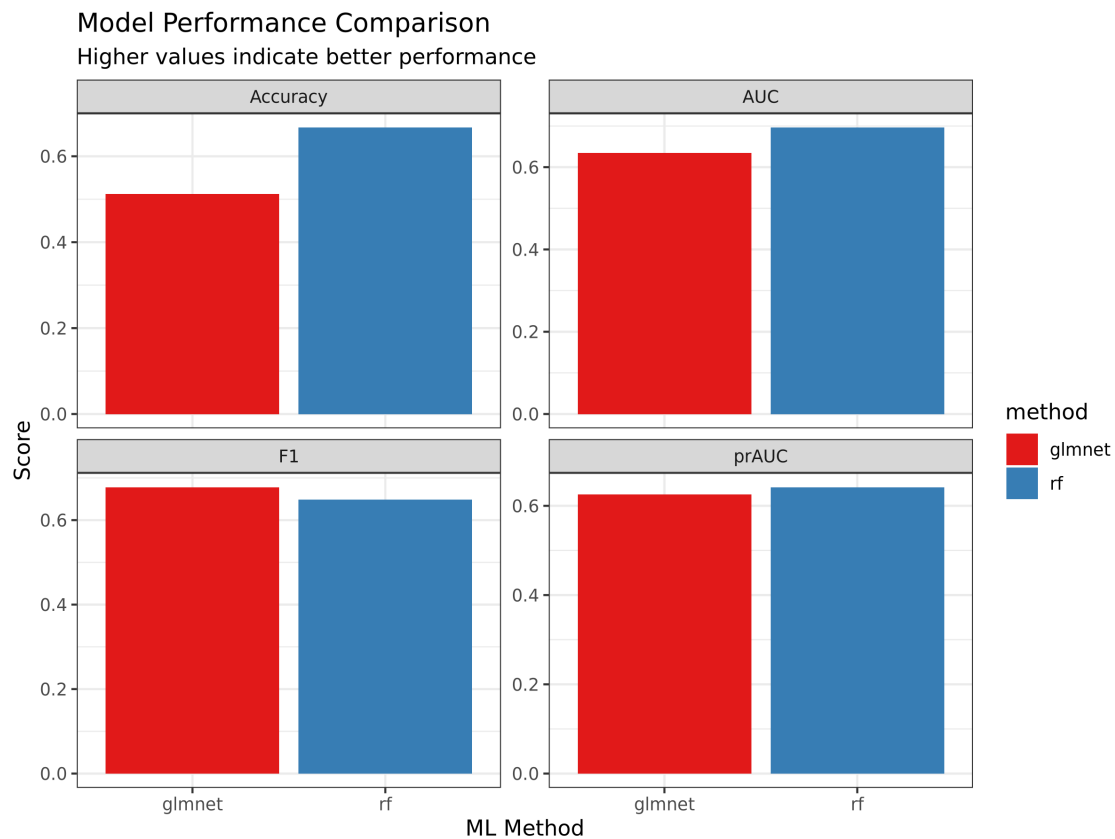
- y-axis: sensitivity
- x-axis: specificity, reversed
- each point is a classification threshold
- better models move toward high sensitivity and high specificity

AUC:

- 0.5: no better than random ranking
- 1.0: perfect ranking

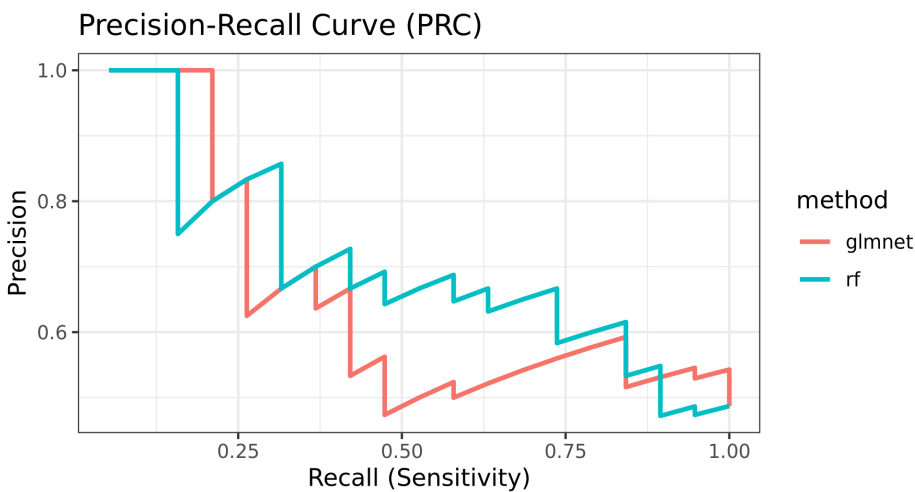


Script 03: Compare Models Visually



RF improves AUC and accuracy relative to the linear baseline.

Decision point:



Precision-recall curves focus on the positive class tradeoff.

Script 04: Hyperparameter Tuning

Hyperparameters are model settings chosen before fitting.

```
1 # 04_hyperparameter_tuning.R lines 42-52
2 custom_hp_glmnet <- list(
3   alpha = 0,
4   lambda = c(0.0001, 0.001, 0.01, 0.1, 1)
5 )
6
7 custom_hp_rf <- list(
8   mtry = c(2, 5, 10)
9 )
```

```
1 # 04_hyperparameter_tuning.R lines 57-74
2 tuned_rf <- run_ml(
3   dataset = input_data,
4   method = "rf",
5   hyperparameters = custom_hp_rf,
6   preProcess = model_preprocess,
7   cv_times = 5
8 )
```

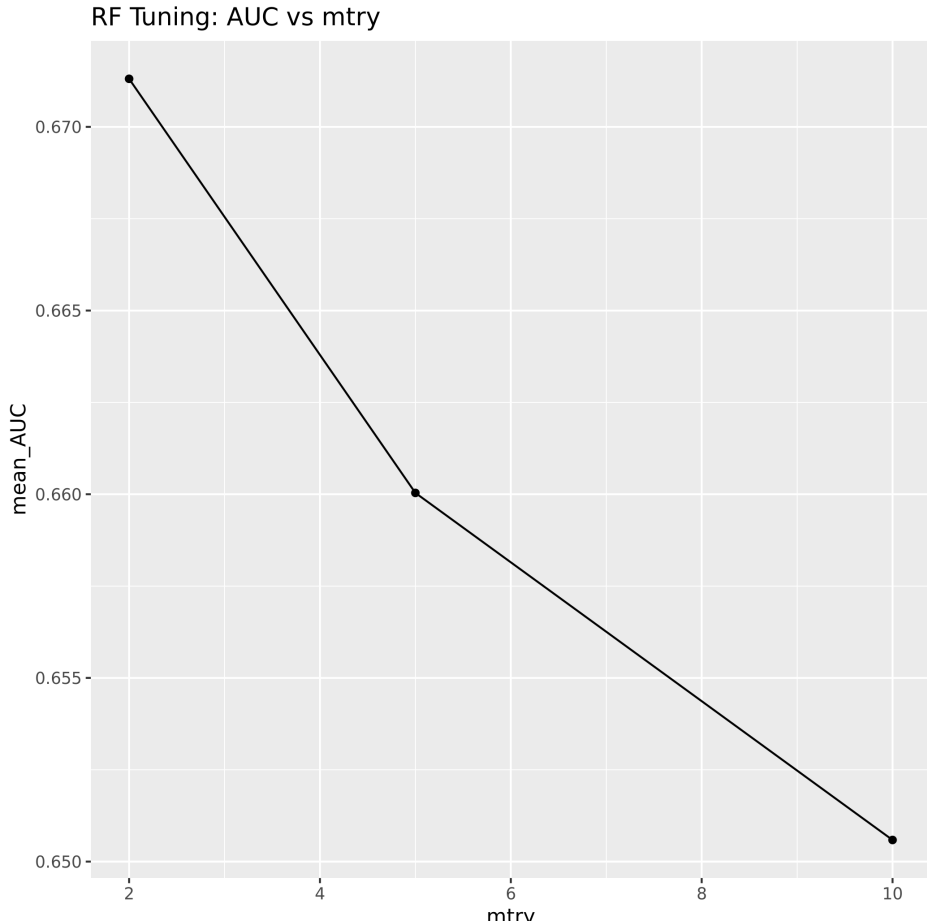
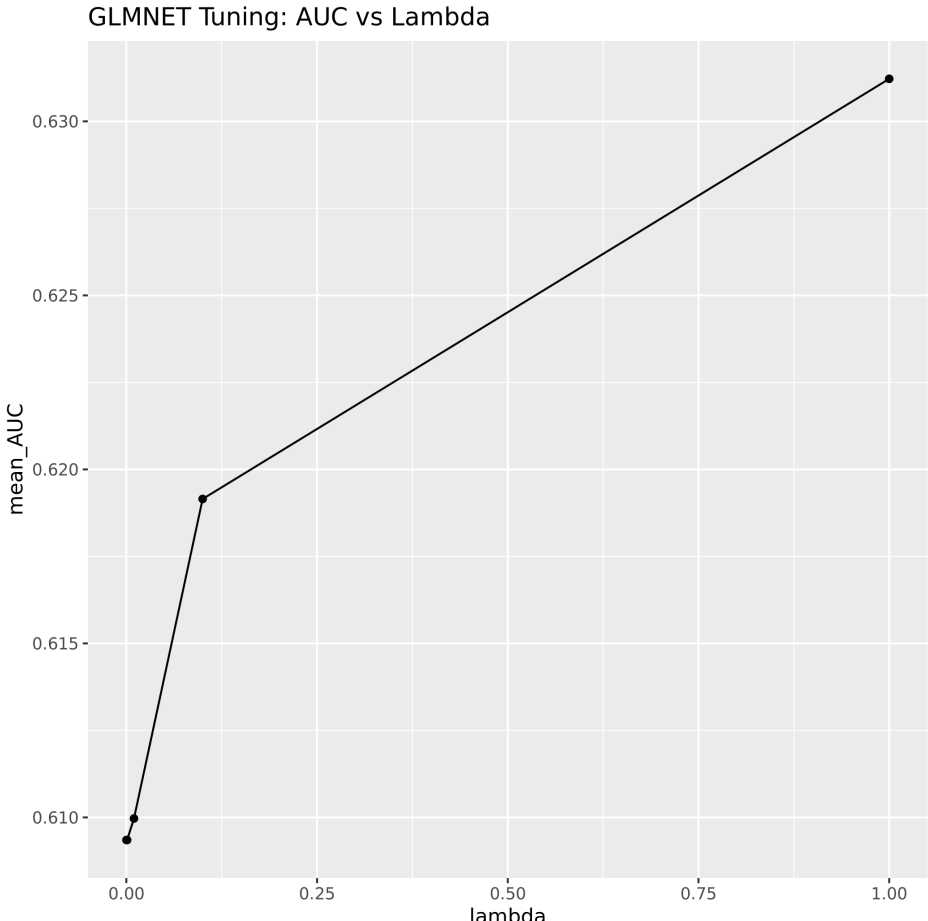
Mathematical intuition:

- **lambda**: regularization strength; larger values shrink coefficients more
- **mtry**: number of features considered at each tree split

Tuning tests several hyperparameter values using cross-validation, then evaluates the selected model on held-out data.

Script 04: Tuning Outcomes

Method	Version	Test AUC	prAUC	Accuracy
glmnet	Default	0.634	0.625	0.513
glmnet	Tuned	0.632	0.621	0.564
rf	Default	0.696	0.642	0.667
rf	Tuned	0.717	0.658	0.718



Script 05: The Black Box Problem

Some ML models, especially ensembles, are often called “black boxes” because:

- many internal rules contribute to each prediction
- nonlinear interactions can be hard to summarize
- high prediction performance does not automatically explain biology

Random forests are more interpretable than a pure black box if we ask:

- Which feature changes affect model performance most?
- Does shuffling an OTU damage prediction?
- Do simpler models agree on directionality?

Interpretation is not proof of mechanism. It is a way to generate biological hypotheses.

Script 05: Permutation Importance

```
1 # 05_feature_importance.R lines 44-72
2 for (f in features) {
3   perm_data <- test_data
4   perm_data[[f]] <- sample(perm_data[[f]])
5   perm_auc <- twoClassSummary(...)
6   importance <- baseline_auc - mean(perm_aucs)
7 }
```

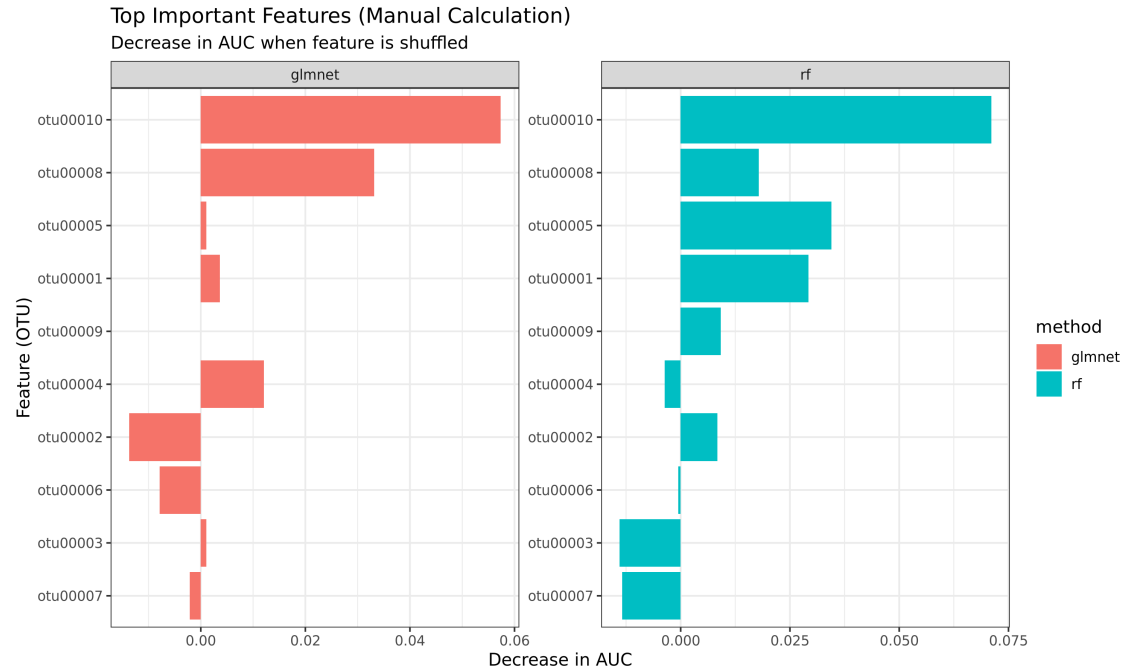
Mathematically:

1. Measure baseline AUC.
2. Shuffle one OTU column, breaking its sample-level relationship.
3. Recompute AUC.
4. Large AUC drop means the model relied on that OTU.

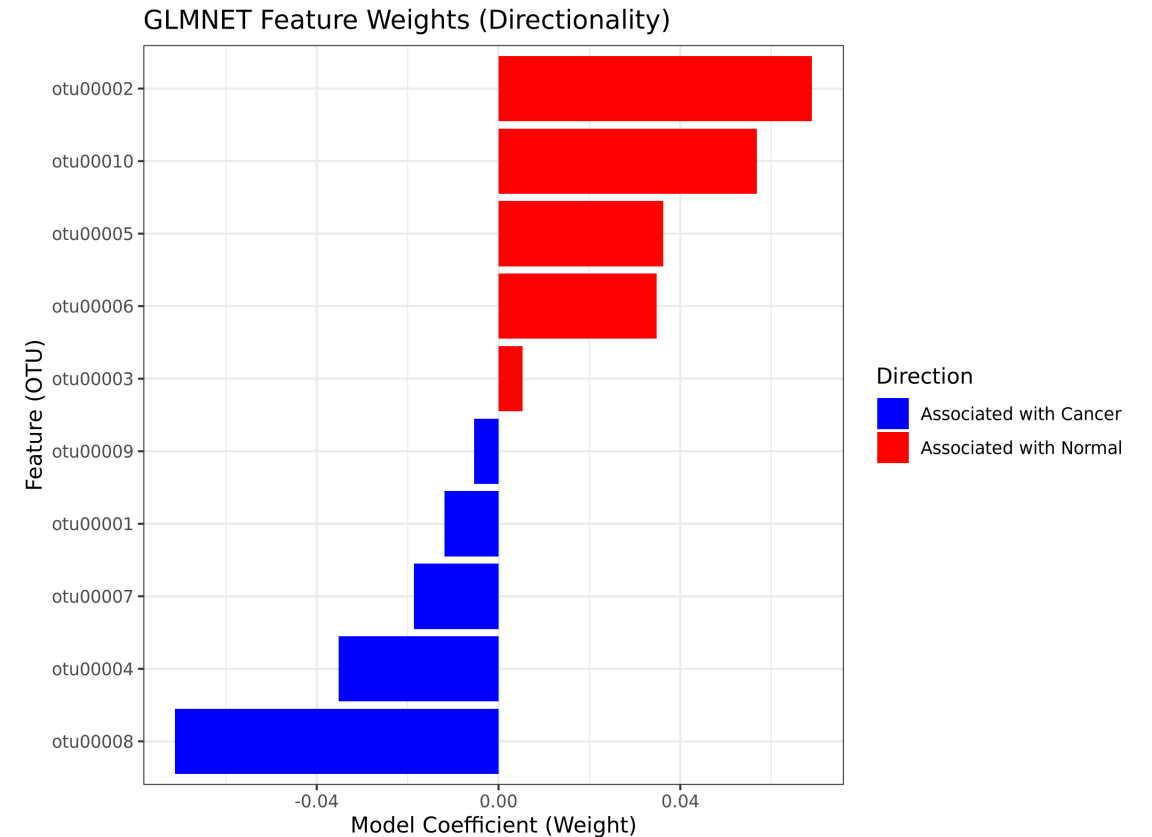
Top current RF features:

Rank	Feature	AUC drop
1	otu00010	0.071
2	otu00005	0.034
3	otu00001	0.029

Script 05: Open the Box



Permutation importance: which shuffled OTUs damage AUC most?



GLMNET coefficients: direction and sign of linear associations.

```
1 # 05_feature_importance.R lines 103-107
2 coefs <- as.matrix(coef(final_model, s = best_lambda))
3 coef_df <- data.frame(feats = rownames(coefs), weights = coefs[, 1])
```

Script 01b: Intentional Error Diagnostics

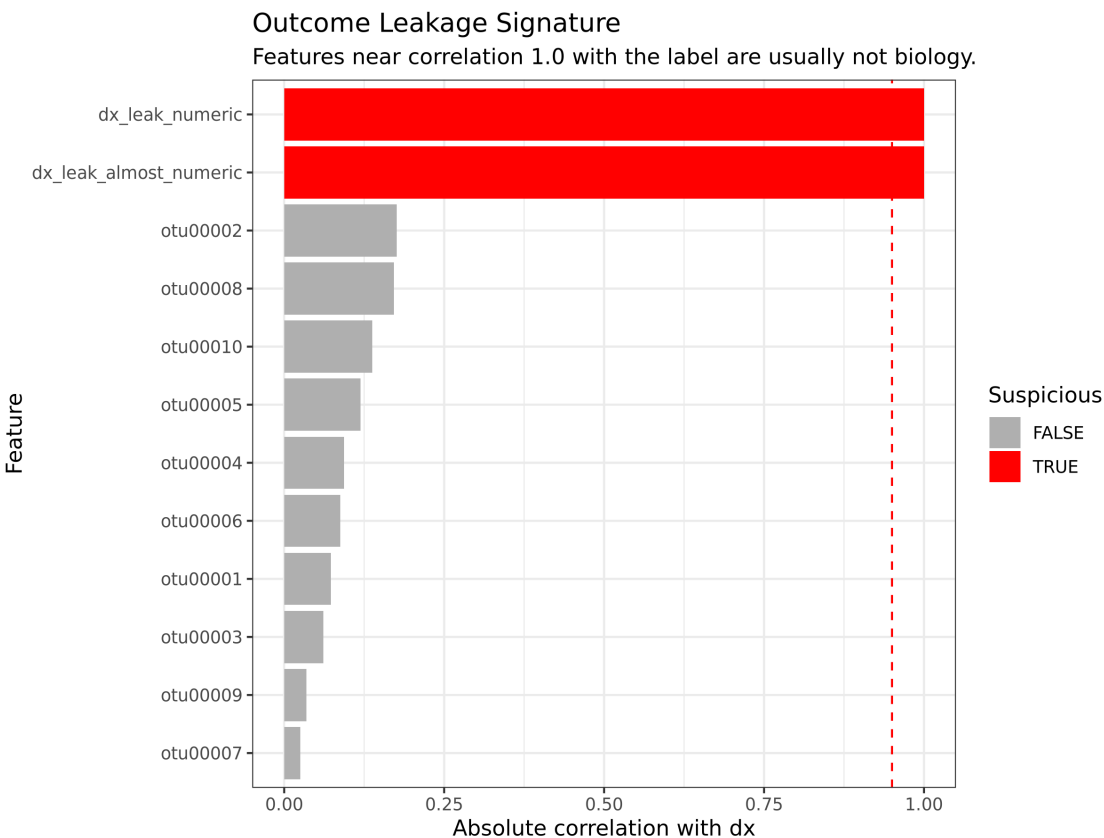
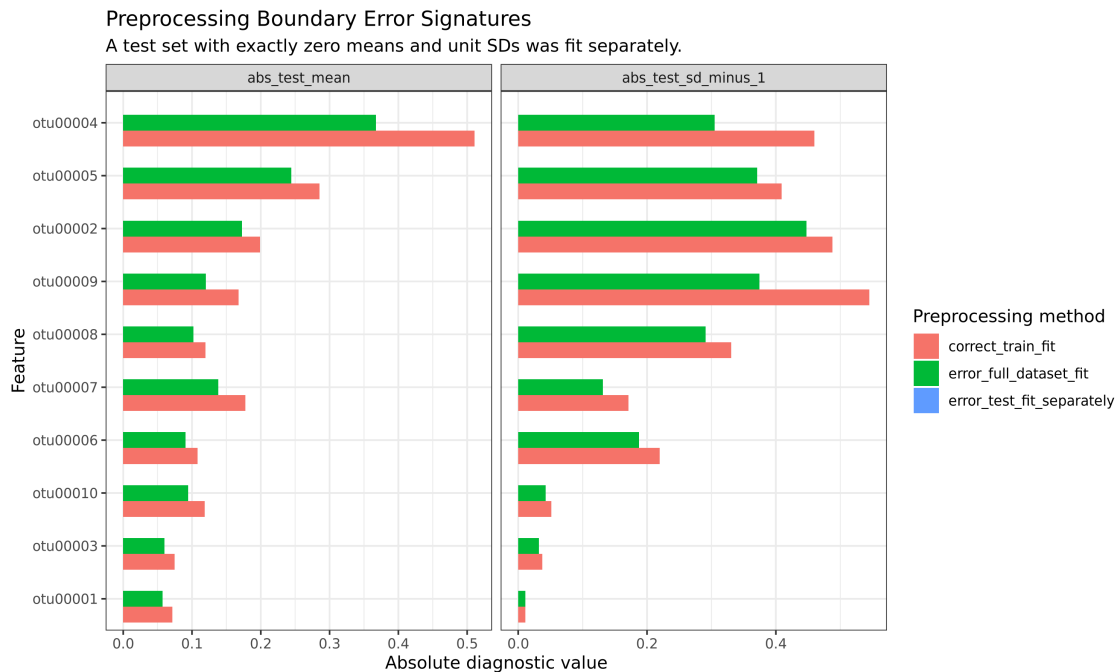
01b_data_preparation_witherrors.R is not part of the main pipeline.

It creates diagnostic examples so students can recognize what wrong workflows look like.

```
1 # 01b_data_preparation_witherrors.R lines 85-96
2 train_centers <- vapply(train_raw[feature_cols], mean, numeric(1))
3 full_centers <- vapply(prepared_data[feature_cols], mean, numeric(1))
4 test_centers <- vapply(test_raw[feature_cols], mean, numeric(1))
5
6 correct_test_scaled <- scale_using(test_raw, train_centers, train_scales)
7 bad_full_data_scaled <- scale_using(test_raw, full_centers, full_scales)
8 bad_test_fit_scaled <- scale_using(test_raw, test_centers, test_scales)
```

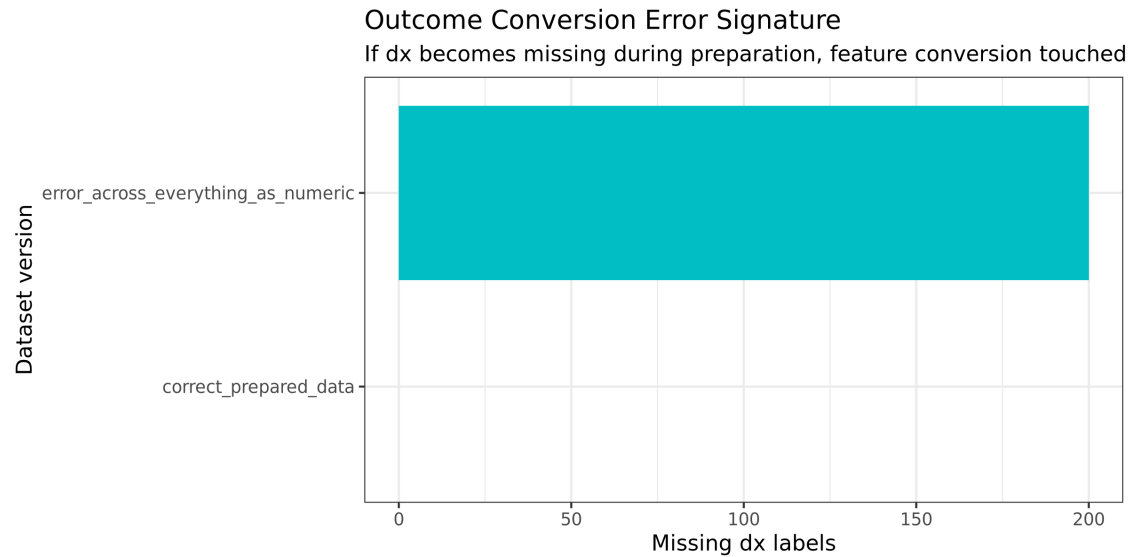
Do not feed `witherrors_*` outputs into scripts 02 through 05.

Script 01b: Failure Signatures

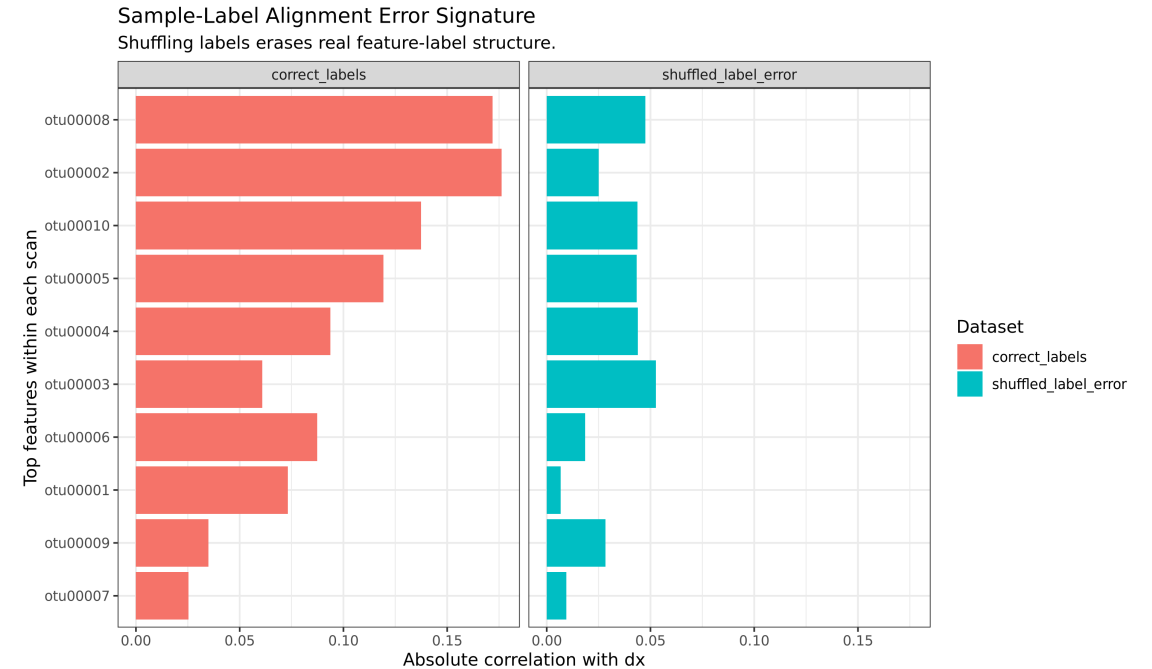


Diagnostic	Bad signature
Test set scaled separately	test means exactly 0 and SDs exactly 1
Outcome leakage feature	correlation with <code>dx</code> near 1.0
Outcome conversion error	all <code>dx</code> values become missing
Shuffled labels	feature-label signal collapses

Script 01b: More Failure Signatures



Bad numeric conversion touches **dx**, turning labels into missing values.



Shuffled labels reduce feature-label structure because samples no longer match outcomes.

Inspect the CSV outputs:

- [results/witherrors_outcome_conversion_diagnostics.csv](#)
- [results/witherrors_label_alignment_summary.csv](#)

Correct vs Errored Workflows

Topic	Correct scripts	Errored diagnostic script
Learned preprocessing	<code>preProcess</code> inside <code>run_ml()</code>	shows full-data/test-fit scaling errors
Outcome handling	<code>mutate(across(-dx, as.numeric))</code>	shows <code>across(everything(), as.numeric)</code> failure
Label integrity	row labels stay attached to samples	shows shuffled-label signal collapse
Leakage features	no label-derived features	adds <code>dx_leak_numeric</code> on purpose
Outputs	<code>prepared_data.rds</code> , model results	<code>witherrors_*</code> only

Exit checklist:

- Can you explain where the train/test boundary is?
- Can you identify which plot shows model discrimination?
- Can you name one OTU the RF model uses?
- Can you recognize an outcome-leakage signature?

Good ML practice is less about one algorithm and more about honest data boundaries, reproducible evaluation, and skeptical interpretation.